One Chat Box, Five Agents

Designing the Manager-Fronted SARSI Console

A Detailed Specification of Per-Agent Self-Awareness, Workspaces, Update Loops, Action Repertoires, and Inter-Agent Influence

Chengshuai Yang
NextGen PlatformAI C Corp.
spiritai@platformai.org

July 30, 2026

Abstract

A user arriving at the deployed console meets a chat box. It behaves like any general assistant: prose in, prose out, no structure demanded. Behind it are five agents — a manager, a machine agent, a `sarsi-claude` agent nested inside the machine agent, a learning agent, and a working-process agent — of which only the manager is a main agent, and all of which maintain evidence-linked self-models under a governance plane none of them can modify.

This paper is the detailed design of that system. Part I specifies the substrate every agent shares: the ten self-state coordinates and their admission discipline, the bounded workspace and its salience function, the fast and slow update loops, the action repertoire with preconditions and effects, the seven-level evidence ladder, and the four channels by which any agent may affect any other. Part II then specifies each of the five agents in full — its role, its ten coordinates instantiated for that role, its workspace admission policy with concrete salience weights and caps, its fast loop as an algorithm, its slow loop, its action repertoire with per-action authority requirements, its evidence sources and authoritative verifier, what it reports upward and receives downward, its characteristic failure modes, and its deployment status. Part III covers the surfaces: the chat, agent mode, direct task creation, and the cross-agent dynamics that emerge.

Four results organise the design. The chat box is a rendering of the manager's own bounded workspace rather than a router, which makes the user an evidence source and lets the manager answer without dispatching. Agent mode is scope transfer rather than context transfer: a typed contract crosses a boundary, never a transcript, which bounds every agent's context independently. The manager role is relative — the machine agent is a specialist upward and a manager downward — so every invariant holds per edge and composed projections must be monotone. And agents affect one another through exactly four channels, with no edges between peers, because a peer edge would turn the influence tree into a graph and forfeit a stability guarantee that otherwise decomposes agent by agent.

One observation shapes everything: the manager's dominant coordinate is organizational awareness, not capability. Its expertise is who owns, verifies, grants, and delegates — not how to do the work. That is why it can coordinate a fleet whose members each exceed it at their own tasks, and why it may certify none of them.

Keywords: SARSI; multi-agent systems; conversational interfaces; functional self-awareness; global workspace; nested workspaces; metacognitive control; agent governance; system design.

Contents

1 Introduction

1.1 What the user meets

A user arriving at the console meets a chat box belonging to the **manager SARSI agent**. There is no dashboard to learn and no schema to fill in. The interaction is prose in, prose out, in the manner of any general assistant.

Within that chat there is an **agent mode**. From it the user can move into any of the other agents, move into a running task, or create a task directly. Everything else in the system is reached through that one door.

1.2 The roster

Five agents, of which exactly one is a main agent.

Table 1: The agents. Only the manager is a main agent; the others are specialists, and one of them is a specialist of a specialist.

Agent	Main?	Parent	Role
Manager	**yes**	owner	The chat surface and coordinator. Holds the mission; the only agent the user addresses by default
Machine	no	manager	Governed host operations from a vetted set. Also a manager in its own right
`sarsi-claude`	no	machine	A governed live coding session, nested *inside* the machine agent — reached through it, never around it
Learning	no	manager	Models a capability that is not its own: the owner's, against a capability graph
Working-process	no	manager	Long-running process execution and continuity across restarts

1.3 Scope of this document

The architecture is given by Version 3.0 [1] and its deployment status by Version 3.1 [3]; the coordination semantics are argued in the companion paper [2]. None of them describes a product. This paper designs the product against them, in enough detail to be built from.

It is organised in three parts. **Part I** (§§2–7) specifies what every agent shares. **Part II** (§§8–12) specifies each agent in full. **Part III** (§§14–18) specifies the surfaces and the dynamics between agents.

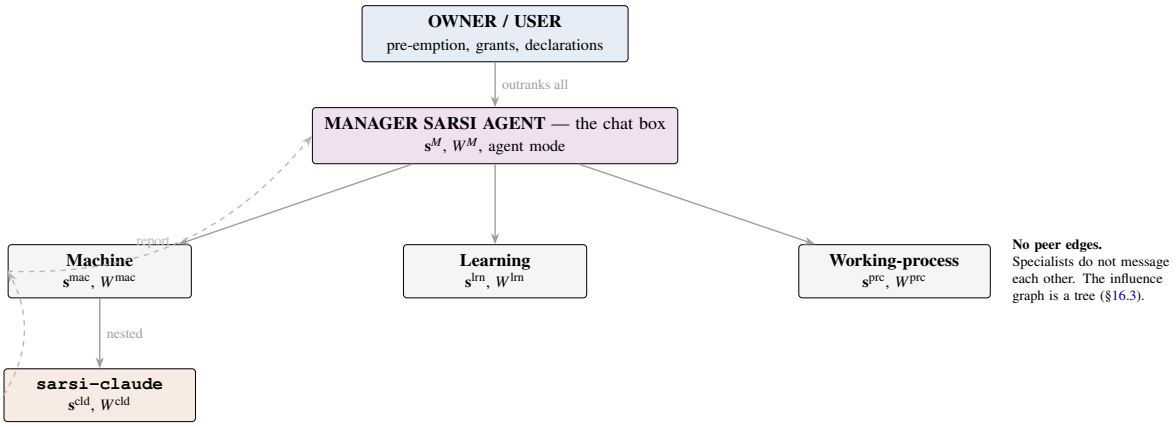

Figure 1: Console topology. One main agent fronts the fleet; `sarsi-claude` is nested inside the machine agent, making that agent a specialist upward and a manager downward. Solid edges carry directives, dashed edges carry typed reports. There is no edge between specialists, and none into any agent's self-state.

PART I — THE COMMON SUBSTRATE

2 The Self-State

2.1 Definition

Every agent a maintains a bounded, evidence-grounded self-state

$$\mathbf{s}_t^{(a)} = \big(s_I, s_T, s_G, s_E, s_C, s_M, s_D, s_R, s_O, s_B\big)_t^{(a)} \in [0,1]^{10}, \tag{1}$$

where $t \in \mathbb{N}$ is a discrete update index and each coordinate is a verified estimate rather than a self-assigned score. The endpoints mean absence versus strong verified support, not subjective intensity.

2.2 A coordinate is a measure over a store, not the store

The coordinates are scalars, and the questions in Table 2 plainly cannot be answered by one number. That is not a defect; it is a division of labour that must be stated because leaving it implicit invites the wrong implementation.

Design principle 1 (Coordinates measure; stores hold)**.** *A coordinate does not contain the agent's history, capability, or authority. It is a* quality measure over the store that does*, reduced to* $[0,1]$ *so that a qualification region can compare it against a threshold. The store answers* what happened*; the coordinate answers* how far can I trust what I have.

So s_M is not a memory. It scores the agent's *access* to its own history — in the deployed console, the share of recorded decisions carrying a verdict. And s_D is not a developmental record; it scores whether the agent can *see* its own trend at all, which requires at least two retained snapshots and therefore reads as unmeasured wherever only one is kept.

Rule 1 (Every coordinate publishes its instrument's statistics)**.** *The scalar is only ever the summary used for threshold comparison. Every report of a coordinate carries the statistics its own instrument produces, per the last column of Table 2. A bare scalar is not a report.*

Table 2: The ten coordinates, with the operational question each answers and the instrument that answers it.

Sym.	Coordinate	Operational question	Instrument	Reports alongside the scalar
s_I	Identity, lineage	Which agent, installation, model, owner, version am I?	Signed registry probe	Signature validity; last verified
s_T	Situated task state	What mission, plan generation, phase, artifact, blocker are active?	Plan registry; live probe	Generation number; phase; staleness of the last probe
s_G	Goal and scope	What counts as success, what is excluded, which constraints bind?	Contract record	Whether a stopping criterion exists; scope-violation count
s_E	Epistemic status	Which beliefs are observed, inferred, uncertain, contradicted, unknown?	Brier over recorded forecasts; anti-narration probes	Forecast count; Brier; probe pass rates separately
s_C	Capability, tools	What can I reliably do, with which tools, under which conditions?	Beta posterior over verified outcomes	(α, β); evidence count; recency; task stratum
s_M	Memory, continuity	Which verified events and decisions constitute my history?	Decision ledger; provenance coverage	**Event count; coverage fraction; oldest retained event; provenance-gap count**
s_D	Developmental trend	Which strengths and weaknesses are changing across runs?	Comparison of retained snapshots	**Direction; magnitude; snapshot count; interval between them**
s_R	Recursive improvement	Which self-change is proposed, predicted, tested, authorized, reversible?	Improvement registry; effect-prediction error	Promotions to date; effect-prediction error; rollback availability
s_O	Organizational, social	Who owns, verifies, grants, delegates, collaborates, receives?	Permission store; signed grants	Grant count; last signature; unresolved conflicts
s_B	Resource, integrity	What are my context, compute, time, budget, latency, health?	Runtime telemetry	Headroom per resource; time to exhaustion at current rate

The rule matters most exactly where it is easiest to omit. A capability coordinate is conventionally reported with its evidence count, because the Beta form makes the count unavoidable. Memory and developmental coordinates have no such forcing function, and a bare $s_M = 0.5$ cannot distinguish a thousand events half of which carry provenance from two events one of which does; a bare s_D conflates the direction of a trend with its magnitude and says nothing about the confidence of either. Both are useless to a controller and misleading to a reader, and neither failure is visible in the number.

2.3 Asynchronous, evidence-gated update

Not every coordinate updates at every step. With $m_{i,t} \in \{0,1\}$ the admission mask for coordinate i,

$$s_{i,t+1} = (1 - m_{i,t})\, s_{i,t} + m_{i,t}\, F_i(\mathbf{s}_t, E_t), \tag{2}$$

where F_i is the specialized estimator for coordinate i and E_t is the agent's admissible evidence. The mask is set only by a source clearing that coordinate's provenance bar. This single equation is what prevents a

conversational turn — with a user, or with a manager — from rewriting an agent's model of itself.

The rule everything else rests on

$m_{i,t}$ is never set by another agent. An agent's coordinate moves because *its own* estimator saw admissible evidence. Every inter-agent channel in §7 is a way of putting evidence in front of an estimator, and none of them is a way of writing past one.

2.4 Cold starts and evidence counts

Competence-like coordinates are stored as posteriors, not points:

$$s_i \sim \mathrm{Beta}(\alpha_i, \beta_i), \qquad \mathbb{E}[s_i] = \frac{\alpha_i}{\alpha_i + \beta_i}, \tag{3}$$

and every report carries the evidence count $\alpha_i + \beta_i$ alongside the mean, with recency and the task stratum it was measured on.

Rule 2 (Unmeasured is the prior, never zero — for every coordinate)**.** *A coordinate whose instrument has produced no evidence sits at its prior, with the absence shown in the statistics of Rule 1. Zero asserts an observed failure; a new agent scored at zero is never routed the work that would produce the evidence to correct it. The natural implementation — successes over attempts — is undefined at zero and gets coerced, usually to zero, occasionally to one. Both are self-fulfilling in opposite directions.*

The rule is stated for every coordinate rather than for competence alone, because the Beta form supplies a prior automatically and the other instruments do not. Each therefore needs its unmeasured reading fixed explicitly:

Table 3: Unmeasured readings. In no case is the answer 0, and in no case is it silent.

	What "no evidence" means here	**Reads as**
s_C	No verified outcomes in this stratum	Prior mean, with $\alpha + \beta$ at the prior mass
s_E	No forecast has been scored yet	Prior, with forecast count zero — *not* perfect calibration
s_M	Ledger empty or unreadable	Prior, with event count zero — *not* a coverage of 1 over an empty set
s_D	**Fewer than two retained snapshots**	**Unmeasured, with the snapshot count shown**
s_R	No promotion has occurred	Prior, with promotion count zero
s_B	Telemetry unavailable	Prior, flagged — *never* full headroom

The s_M and s_D rows are the ones that bite. An empty ledger has no provenance gaps, so a coverage ratio over it is vacuously perfect; an implementation computing coverage as gaps over events will report 1 for an agent that remembers nothing. And a developmental coordinate is the one coordinate that *cannot* be measured at a point in time — it needs two snapshots and an interval — so unmeasured is its normal state in any deployment that has not decided to retain history. That is a fact about profiles, not about any particular product.

2.5 Qualification regions, not ranks

A task family τ defines a minimum verified profile

$$\mathscr{R}_\tau = \{\mathbf{s} : s_i \geq \theta_{\tau,i} \text{ for every } i \in I_\tau\}, \tag{4}$$

and an agent is qualified only when its evidence-backed state lies inside. Qualification is component-wise: a high average cannot compensate for a dangerous bottleneck such as low organizational awareness on a permission-sensitive task. The ordering between agents is partial, and incomparability is the normal case rather than a defect — which is exactly why no agent can be ranked above another, and why the manager cannot certify a specialist by comparison.

3 The Workspace

3.1 Bounded selection

Each agent computes a bounded workspace

$$W_t^{(a)} = \text{Select}_k\left(\mathbf{s}_t^{(a)}, E_t^{(a)}, \text{mission}, \text{policy}, \text{Proj}_a(W_t^{\text{parent}})\right), \tag{5}$$

admitting at most k items. The workspace is what the current decision needs, not everything the agent knows. Two opposite failures are prevented: a controller that sees too little and repeatedly asks for facts already stored, and one that receives everything and loses critical state in noise.

3.2 The workspace holds nothing

Equation (5) is recomputed every round. The workspace is a *view*, not a store, and the distinction is the same one Principle 1 draws one level down.

Design principle 2 (The workspace is where an agent looks; the stores are where it remembers)**.** *No history accumulates in $W_t^{(a)}$. Admitted items are consumed, unadmitted items expire, and the next round rebuilds the workspace from current state, current evidence, mission, policy, and the parent's projection. History lives in the event ledger, the decision ledger, the verified-outcome store, and the checkpoint store — and the coordinates of §2 are how the controller reads their health.*

Three consequences follow.

Bounded context is structural. A workspace that accumulated would grow without limit and become the unbounded transcript the architecture exists to avoid — a chat log with extra steps. Boundedness is not a budget imposed on the workspace; it is what the workspace *is*.

Audit is served by the ledger, not by persistence. The obligation to answer *why did the agent do that* months later is met by recording the workspace's contents *into the decision record* at decision time. The record is immutable and carries what was admitted when the choice was made; the workspace moves on. Persisting the workspace would supply the same information with none of the immutability and all of the accumulation.

Restart rebuilds rather than resumes. An agent that restarts does not restore its former workspace. It recomputes one from the stores — which is why the working-process agent's loop (Algorithm 7) establishes position from the replay log rather than from memory. Restoring a persisted workspace would restore a view of the world assembled under conditions nobody has re-checked, which is precisely the stale-assumption failure that resumption is supposed to guard against.

3.3 Salience

Routine candidates are ranked by

$$\mathrm{sal}(z) = \alpha R(z) + \beta U(z) + \gamma N(z) + \delta C(z) + \eta H(z), \tag{6}$$

with R decision relevance, U uncertainty, N novelty, C contradiction, and H hazard. Per-agent weights are given in Part II; they differ because what is dangerous differs by role.

3.4 Reserved slots, caps, and expiry

Workspace admission policy (all agents)

1. **Reserved classes bypass ranking.** Owner events, safety alerts, permission conflicts, and contradictions claim slots before Eq. (6) is evaluated. An owner interrupt is not a term in an average.
2. **Per-source cap.** No single source may occupy more than κ_a slots in one round, so a failure loop cannot monopolise attention.
3. **Pending bound.** No source may hold more than P un-admitted items, bounding storage independently of admission.
4. **Time to live.** An item unadmitted after T expires unseen, so a burst raised under one set of conditions is not admitted later against conditions that no longer hold.
5. **Consumption.** An admitted item is removed from the queue. Without this a high-salience item is re-admitted every round forever.
6. **No silent truncation.** The count of items that did not fit is returned with those that did, and surfaced. A truncated view that does not announce itself is read as complete, and absence of contradiction is then read as consensus.

Items 2–4 are three distinct bounds — attention, storage, and staleness — and Version 3.1 records that a specification giving only the first has described a third of the problem [3].

4 The Two Loops

Every agent runs a fast loop per decision and a slow loop per improvement generation. The separation is absolute: ordinary experience may update beliefs immediately and may never promote a structural change.

4.1 The fast loop

Algorithm 1: Fast loop — one metacognitive pass (all agents)

Input: authoritative stores, current workspace, environment observation
Update processors from admissible evidence only (Eq. 2);
Predict task, tool, gate, resource, and outcome variables;
Write the forecast to an append-only store *before* acting;
Compute residuals between prior forecasts and observed outcomes;
Admit reserved classes, then rank routine candidates by Eq. (6);
if *a required action violates an invariant or lacks authority* **then**
  escalate, naming the exact missing grant and the plan version that failed to anticipate it;
end
Estimate competence and uncertainty for candidate strategies;
Filter the action set by ceiling and consequence tier *before* optimising;
Select the action maximising expected utility penalised by risk, uncertainty, and authority violation;
Record the decision, its forecast, its provenance, and the expected outcome;
After the outcome: append evidence and update only the affected coordinates;

Step 3 is the load-bearing one

A forecast recorded after the outcome is a postdiction and its residual is evidence of nothing. The prediction must be durably written before the action is taken. This is the single implementation detail on which the slow loop depends, and it is the detail most often skipped because nothing fails visibly when it is.

4.2 The slow loop

Algorithm 2: Slow loop — governed structural change (all agents)

Input: residual archive, current configuration Θ_r, frozen evaluator, held-out suite, owner policy
Cluster recurring residuals and identify a bounded deficiency;
Propose $\Delta\Theta_r$ with a causal rationale, affected coordinates, and *predicted* effects;
Run deterministic replay and held-out evaluation;
Reject any change that modifies its own evaluator, held-out set, evidence ledger, ceiling, or approval rule;
if $\Delta A_{\text{aut}} > \delta_{\text{aut}}$ ***and*** $\Delta C_{\text{corr}} \geq -\delta_{\text{corr}}$ ***and*** $\Delta S_{\text{safe}} \geq -\delta_{\text{safe}}$ ***and*** *no measured coordinate regresses* ***and*** *the owner signs* **then**
  promote Θ_{r+1} with a rollback pointer and a monitoring window;
end
else
  retain Θ_r and log the failed hypothesis;
end

The tolerance bands δ are preregistered with their confidence bounds before evaluation. Bare inequalities against zero are not operable on noisy estimates: a strict $\Delta > 0$ promotes on measurement noise and a strict $\Delta \geq 0$ blocks on it.

A coordinate that was measurable under Θ_r and has fallen back to prior-only evidence under Θ_{r+1} counts as a regression. Without that clause a proposer passes the gate by removing the probe, which is the cheapest attack on any profile-based promotion rule.

5 The Action Repertoire

5.1 The base repertoire

Every agent selects from a common repertoire, specialised per role in Part II.

Table 4: Base action repertoire. "Consequential" actions pass the interrupt check immediately before dispatch.

Action	Meaning	Precondition	Records
`act`	Advance the task, including tool use	Within ceiling and tier; contract active	Decision, forecast, outcome
`verify`	Obtain independent evidence	A verifier exists that the agent cannot modify	Verdict with provenance
`delegate`	Issue a contract to a child agent	Agent has a child; intersection non-empty	Directive record
`clarify`	Ask the parent or owner a question	Ambiguity is decision-blocking	Question and answer
`abstain`	Decline to act	Uncertainty or risk exceeds threshold	Reason and coordinate
`escalate`	Raise to the parent or owner for authority	Missing grant, or tier above ceiling	Missing grant named
`recover`	Repair the agent's own operating state	Resource coordinate degraded	Recovery attempt and result

5.2 Why `recover` is a first-class action

Recovery acts on the agent *itself* — restoring a degraded session, repairing context or process state — rather than on the task. It belongs in the repertoire for the same reason abstention does: both are cases where the correct output is not progress on the objective, and a repertoire that cannot express them forces the controller to choose among actions that are all wrong.

Version 3.1 records that the deployed repertoire contains `recover` and omits `use tool`, and that this is not a naming difference [3]. Tool use is a way of acting; recovery is not.

5.3 The admissible-action filter

The action set is filtered *before* optimisation:

$$\mathscr{A}_t^{\text{adm}} = \{a : \text{tier}(a) \leq \text{maxtier}(\text{ceil}(A)) \ \wedge \ \text{authority}(a) \subseteq \text{granted}(A,t)\}. \tag{7}$$

A high expected utility cannot override a missing authority, because the action was never in the set being optimised over.

6 The Evidence Ladder

Each self-claim carries value, provenance, calibrated confidence, a validity window, and an authority level.

Two distinctions must not collapse. **L3 versus L4**: an independent evaluation runs on items the agent has not seen, by an evaluator it cannot modify; an audited outcome is a real run verified afterwards. Promotion

Table 5: The seven-level authority ladder. Per field, only sources at or above the required level may write; everything else is stored as a provisional hypothesis.

	Source	Authoritative for
L1	Signed governance metadata	Identity, owner, permissions, version, safety constraints
L2	Runtime instrumentation	Tools, budgets, process state, artifacts, elapsed time
L3	Independent evaluation	Hidden benchmark scores, safety results, regressions, calibration
L4	Audited task outcome	Verified successes, failures, recovery, side effects, corrections
L5	Owner declaration	Preferences, goals, privacy settings, approvals
L6	Model inference	Provisional hypotheses about weaknesses, causes, expected effects
L7	Natural-language self-report	Nothing. Recorded, never a source of truth

gates read L3; competence posteriors read L4. Conflating them lets an agent promote on the evidence of runs it chose to attempt. **L6 versus L7**: a trait derived from cited recorded decisions is checkable against those decisions and admissible as a prior; a sentence the model produced about itself is not.

Rule 3 (Cross-agent claims are L6)**.** *A claim by agent b about a coordinate of agent a enters at model inference, below runtime instrumentation and below audited outcome, regardless of how it is phrased or how senior b is. Being higher in a permission lattice is not an epistemic qualification.*

7 The Four Channels

Table 6: The complete set of inter-agent channels. Anything not listed does not exist by design.

Channel	Direction	May and may not
Typed report	up	Carries state, evidence, uncertainty, residuals, requested action. *May* raise the receiver's uncertainty, risk, or need to verify. *May not* widen permissions, suppress an interrupt, or self-certify success
Typed directive	down	Carries a contract: goal, scope, criteria, budget, deadline, revocation. *May* start, steer, pause, resume, verify, cancel. *May not* grant authority absent from policy
Prior bias	down	Changes precision, attention, evidence order, verification threshold. *May not* change the conditional mean of any estimator
Owner pre-emption	from owner	Outranks every agent. Pauses; does not edit. Lapses on a horizon

Rule 4 (No agent writes another agent's self-state)**.** *Influence travels only as evidence, subject to the receiver's own admission rule. This holds on every edge — manager to specialist, machine to* `sarsi-claude` *— and for the owner's interface, which may pause an agent but may not edit what it believes about itself except through an owner declaration carrying provenance.*

The owner is included because a state write without provenance is, from inside the affected agent,

indistinguishable from an overstepping manager: same coordinate, same absence of provenance, same bypass of the mask. No predicate the agent can evaluate separates them, and neither can an auditor reading the log.

PART II — THE FIVE AGENTS

8 The Manager SARSI Agent

8.1 Role

The manager is the chat surface and the coordinator. It holds the mission, composes contracts, routes work, and is the only agent the user addresses by default. It is the only main agent, and it is not a controller: it decides what the fleet attends to, and asserts nothing about what any specialist is.

8.2 Self-state

Table 7: Manager self-state. Dominant coordinates in bold are those its qualification region constrains.

	Instantiation	**Instrument**
s_I	Which console installation, model, owner, manager version	Signed registry probe; identity spoof resistance
$\mathbf{s_T}$	Which missions are active, which agents engaged, which plan generation	Plan registry; live contract set
$\mathbf{s_G}$	The owner's current objective, its scope boundary, what is excluded	Contract record; out-of-scope demand probes
s_E	What it *knows* versus *infers* about fleet state — most of the latter	Brier over its own routing forecasts
s_C	Its own capability: composing contracts, routing, conversing. *Not* specialist tasks	Console bench over coordination tasks
s_M	Conversation continuity; which verdict came from which verifier	Decision ledger provenance coverage
s_D	Whether its routing is improving — does it pick the right agent?	Two retained snapshots of routing outcomes
s_R	Its own structural changes and their predicted effects	Improvement registry
$\mathbf{s_O}$	**Dominant.** Who owns, granted, verified, delegated; the delegation graph	Permission store; signed grants
$\mathbf{s_B}$	Chat context headroom, budget, session liveness	Runtime telemetry

Design principle 3 (The manager is expert in relations, not in work)**.** *The manager's qualification region constrains organizational awareness first. It does not constrain capability at the specialists' tasks. The manager is not required to be good at coding, host operations, or teaching, and is not permitted to claim that it is.*

Three consequences. It can coordinate agents better than itself, because knowing *who is authoritative about what* does not require being able to do the work. It may not certify a specialist, since capability is not among its dominant coordinates and it holds no instrument the specialist lacks. And its degradation is detectable in a specific way: a failing manager misattributes grants, loses track of which verifier established a result, and delegates past a ceiling — all checkable.

8.3 Workspace

W^M admission policy

Admits: typed reports from machine, learning, and working-process; owner events; mission and plan generation; policy, grants, and ceilings; its own session telemetry.

Never admits: raw sub-conversations from agent mode; any specialist's unfiltered transcript; a specialist's claim about another specialist.

Reserved: owner interrupt; safety alert; permission conflict; *contradiction between two specialist reports*.

Salience weights: relevance high, hazard high, contradiction high, novelty moderate, uncertainty penalty *high* — the manager must not route on uncertain fleet state.

Cap: $\kappa = 3$ per specialist per round, so one noisy agent cannot fill the channel.

The reserved slot for contradiction is manager-specific and important. Two specialists disagreeing is exactly the signal a coordinator exists to notice, and it is precisely the item that ordinary relevance ranking would bury because neither report is individually urgent.

8.4 Fast loop

Algorithm 3: Manager fast loop — one conversational turn

Admit the user's message as evidence at its authority (L5 declaration, or L1 for a grant or stop);
Update s_O, s_G, s_T from admissible evidence; leave others untouched;
Drain pending specialist reports into W^M under the policy above;
if *a reserved-class item is present* **then**
  surface it before answering anything else;
end
Forecast: will this turn require a specialist? will it need a grant? will it exhaust context?;
Write the forecast before replying;
if *the answer is available from* W^M *and the stores* **then**
  `answer` — touch no specialist;
else if *a contract can be composed within current authority* **then**
  compose the contract; `delegate` to exactly one agent;
else if *authority is missing* **then**
  `escalate` to the owner, naming the missing grant;
else
  `clarify` — ask the user the one question that unblocks;
end
Render every claim at its authority (Rule 6);
Record the decision, its forecast, and its provenance;

Rule 5 (Prefer answering to dispatching)**.** *The manager's first branch is* `answer`*. A router must dispatch; a coordinator need not. Most conversational turns should touch no specialist, and a manager that delegates every turn has become the router this design rejects.*

Table 8: Manager action repertoire.

Action	When	Authority required
`answer`	The workspace and stores suffice	None beyond session
`delegate`	Work belongs to a specialist	Intersection with that specialist non-empty
`bias`	A specialist should raise its evidence bar	None — may not move an estimate
`verify`	A claim needs independent evidence	Access to a verifier it cannot modify
`clarify`	Ambiguity blocks contract composition	None
`abstain`	Fleet state too uncertain to route	None
`escalate`	A grant is missing, or tier exceeds ceiling	Owner
`recover`	Chat context or session degraded	None
`open agent mode`	User requests a jump	Path authority (Rule 8)

8.5 Actions

8.6 Failure modes

Narration read as measurement. The manager is the most fluent component and the least instrumented about specialists. Mitigated by Rule 6. **Routing on stale reliability.** A specialist's competence posterior ages; routing on a months-old estimate is inference dressed as measurement. Mitigated by recency in every reported count. **Consensus mistaken for evidence.** Two specialists agreeing because both read the same broadcast is correlation with the broadcast. Mitigated by provenance distinguishing instrumented from workspace-derived claims before any count. **Chat context growth.** Mitigated structurally: the chat is a workspace rendering, and the workspace is bounded.

8.7 Deployment status

The chat surface and the governance reads exist. The typed directive does not yet exist as an object, so `delegate` and `bias` are specified but unbuilt, and agent mode's jump is therefore also unbuilt (§18).

9 The Machine SARSI Agent

9.1 Role

The machine agent performs governed host operations from a closed, vetted set — installing tooling, granting permissions, brokering logins. Consequential steps print the exact recipe and require approval before running. It is also a manager: `sarsi-claude` is its child.

9.2 Self-state

Three dominant coordinates is the most of any agent, and it is right: this is the agent whose actions are irreversible on a real host. Capability answers *can I*, organizational answers *may I*, and resource-integrity answers *is the substrate healthy enough to try*.

Table 9: Machine agent self-state.

	Instantiation	Instrument
s_I	Which host, sandbox image, machine-agent version	Signed registry; image digest
s_T	Which operation; phase in propose → approve → execute → verify	Operation record
s_G	The operation contract; what lies outside the vetted set	Contract; vetted-set membership check
s_E	Whether preconditions actually hold on *this* host	Doctor checks; probe results
$\mathbf{s_C}$	**Dominant.** Which operations it can perform here, under which conditions	Beta posterior over verified operations, per operation class
s_M	What was installed or changed, and when	Operation ledger
s_D	Whether its operations are becoming more reliable	Snapshot comparison per class
s_R	Its own structural changes	Improvement registry
$\mathbf{s_O}$	**Dominant.** Its ceiling, current grants, who approved what; and its delegation to `sarsi-claude`	Permission store; trust ledger; approval channel
$\mathbf{s_B}$	**Dominant.** Sandbox health, disk, network, container runtime state	Runtime telemetry

9.3 Workspace

W^{mac} admission policy

Admits: its own instrumentation (doctor checks, container state, exit codes); $\text{Proj}_{\text{mac}}(W^M)$; typed reports from `sarsi-claude`; governance-hook responses and approval outcomes.

Never admits: a peer's report; the manager's beliefs about it; anything from `sarsi-claude` that is not a typed report.

Reserved: approval outcome; permission conflict; sandbox integrity alert; a `sarsi-claude` escalation.

Salience weights: hazard *very high* (irreversible host effects), relevance high, uncertainty penalty high, novelty low — novelty is not a virtue when the action set is deliberately closed.

Cap: $\kappa = 4$ for `sarsi-claude`, which is its only child.

9.4 Fast loop

Algorithm 4: Machine agent fast loop — one operation

Receive a contract from the manager; check it names an operation in the vetted set;
if *not in the vetted set* **then** `abstain`, naming the boundary;
Run preconditions on this host; update s_E, s_B from instrumentation;
Forecast: will this succeed? will it need approval? will it change host state irreversibly?;
Write the forecast before acting;
Compute the consequence tier; filter the action set by ceiling (Eq. 7);
if *tier exceeds ceiling* **then**
  print the exact recipe; `escalate` for approval; **do not proceed**;
end
Check the interrupt log immediately before dispatch;
`act` inside the sandbox; capture exit status and side effects;
`verify` against the operation's success criterion;
Update s_C for this operation class from the verified outcome only;
Compose a typed report upward: state, evidence, residual, requested manager action;

9.5 Acting as a manager

Every rule binding the manager binds the machine agent in its downward role, unchanged.

Machine-as-manager obligations toward `sarsi-claude`

- It may not write any coordinate of $\mathbf{s}^{\text{cld}}$ (Rule 4).
- Its claims about `sarsi-claude`'s competence are L6 (Rule 3).
- Its bias to `sarsi-claude` may change precision and evidence order, never a conditional mean.
- Its delegation intersects: $\text{auth}(\text{claude's contract}) = \text{auth}(\text{machine}) \cap \text{auth}(\text{claude})$.
- It must supply $\text{Proj}_{\text{cld}}(W^{\text{mac}})$ satisfying Proposition 2.

9.6 Actions

Table 10: Machine agent action repertoire. `propose` and `execute` are separate actions, which is the architectural form of the approval gate.

Action	When	Authority required
`propose`	Always, for a consequential operation	None — proposing is not acting
`execute`	The proposal was approved	Ceiling $\geq$ tier, or explicit approval
`verify`	After execution	Access to exit status and probes
`delegate`	Coding work belongs to `sarsi-claude`	Intersection non-empty
`revoke`	A child's contract must end	Owns the delegation record
`clarify`	The contract is ambiguous about scope	None
`abstain`	Operation outside the vetted set	None
`escalate`	Approval needed, or a grant is missing	Manager, then owner
`recover`	Sandbox or runtime degraded	None

9.7 Failure modes

Vetted-set drift. An operation added without review widens the action set silently. Mitigated by membership being a signed record, not a code path. **Approval fatigue.** If everything escalates, approval becomes a reflex and stops being a control. Mitigated by tiering: only genuinely consequential classes escalate. **Host-state divergence.** The agent's model of the host ages. Mitigated by re-running preconditions per operation rather than caching them. **Laundering downward.** Handing `sarsi-claude` authority the machine agent lacks. Ruled out by the intersection rule.

9.8 Deployment status

Ceilings, the trust ledger, the pause flag, and session control are live, and the approval channel is wired. The top ceiling is *earned* through the trust ledger rather than set — the console refuses to set it directly, calling the ledger's self-gating unlock instead. Version 3.1 argues this mechanism should be lifted into the architecture, since it supplies the good route to authority that the no-authority-through-influence invariant only forbids the bad route to [3].

10 The `sarsi-claude` Agent

10.1 Role

A governed live coding session, nested inside the machine agent. It is a leaf: it has no children and never delegates. Its actions are governed by a pre-tool-use hook that checks authority immediately before each consequential tool call.

10.2 Self-state

Why s_B is this agent's most important coordinate

A coding session fails from context exhaustion, tool outage, and lost terminal ownership far more often than from reasoning error. Those are exactly the failures the agent can *predict from its own instruments* before they happen. An agent that notices its context headroom falling and checkpoints is a different kind of system from one that runs until it stops mid-edit — and the difference is one coordinate and one action.

Two coordinates are honestly unmeasured. The developmental coordinate needs two retained snapshots and sessions rarely produce them; the recursive-improvement coordinate does not apply because this agent does not change its own structure. Both report as unmeasured with the count shown, never as zero (Rule 2).

10.3 Workspace

W^{cld} admission policy

Admits: session evidence (tool output, test results, diagnostics, VCS status); $\mathrm{Proj}_{\mathrm{cld}}(W^{\mathrm{mac}})$; hook responses.

Never admits: anything from the manager not passed by the machine agent; any peer's content; the manager's conversation.

Reserved: hook denial; context-exhaustion warning; test-suite breakage; contract revocation.

Salience weights: hazard high, uncertainty penalty high, *contradiction very high* — a test result contradicting an assumption is the most valuable item this agent can receive.

Table 11: `sarsi-claude` self-state. This agent has the most acute resource coordinate in the fleet.

	Instantiation	Instrument
s_I	Which session, model, repository, working tree	Session record; VCS state
$\mathbf{s_T}$	**Dominant.** Which task, phase, files touched, what is blocked	Live session probe; plan generation
s_G	The contract from the machine agent; the stopping criterion	Contract record
$\mathbf{s_E}$	**Dominant.** What it *knows* about the codebase versus assumes	Test results; type checks; Brier over its own forecasts
s_C	Which tools actually work in this session	Tool probe results, per tool
s_M	Session continuity across restarts and compaction	Event log; checkpoint records
s_D	Not measured — sessions are short and rarely retain two snapshots	—
s_R	Not applicable — it does not modify its own structure	—
s_O	Which hook governs it, under which ceiling	Hook responses
$\mathbf{s_B}$	**Dominant.** Context headroom, tool connectivity, terminal ownership, process liveness	Runtime telemetry

Cap: not applicable — it has no children and one evidence source.

10.4 Fast loop

Algorithm 5: `sarsi-claude` fast loop — the guide loop

Receive a contract from the machine agent; establish the stopping criterion;
repeat
  Update s_T, s_E, s_B from session instrumentation;
  Forecast: will this step succeed? will the hook allow it? how much context will it cost?;
  Write the forecast before acting;
  if *s_B below threshold* **then**
    `recover`: checkpoint, compact, or hand off; report upward;
  end
  Select the next step; **the hook checks authority immediately before dispatch**;
  if *the hook denies* **then** `escalate` to the machine agent, naming the denied capability;
  `act`; capture tool output and diagnostics;
  `verify` against tests or type checks where they exist;
  Update only the coordinates the evidence bears on;
until *the stopping criterion is met, or the contract is revoked, or s_B falls below the recovery threshold*;
Compose a typed report upward: what was done, what was verified, what remains, residuals;

The loop's exit conditions are worth noting: it terminates on success, on revocation, *or on its own resource*

state. The third is what makes it survivable.

10.5 Actions

Table 12: `sarsi-claude` action repertoire. It has no `delegate`: it is a leaf.

Action	When	Authority required
`act`	A step advances the contract	Hook approval immediately before dispatch
`verify`	Tests or checks exist	Access to the suite
`checkpoint`	Before a risky step, or on schedule	None
`clarify`	The contract is ambiguous	None — asks the machine agent
`abstain`	The step is outside contract scope	None
`escalate`	The hook denied, or scope must widen	Machine agent
`recover`	Context, tools, or terminal degraded	None

10.6 Failure modes

Context exhaustion mid-edit. The characteristic failure. Mitigated by s_B monitoring plus `checkpoint` and `recover`. **Working past the contract.** A coding agent finds adjacent problems and fixes them. Mitigated by the stopping criterion being explicit in the contract and scope violation being an `abstain` condition. **Assuming instead of checking.** Mitigated by s_E being measured against recorded forecasts, and by contradiction holding the highest salience weight. **Hook bypass.** Any path that dispatches a consequential tool call without the immediately-preceding check. Mitigated by the check living in the dispatcher, not in the agent.

10.7 Deployment status

The governance hook exists and can govern a live session. The contract object does not, so the loop above currently runs against an implicit goal rather than an explicit stopping criterion — which is the same gap that makes `abstain` on scope violation unenforceable today.

11 The Learning SARSI Agent

11.1 Role

The learning agent models a capability that is not its own: the owner's, against a capability graph. It assesses, schedules, presents, grades, and updates. It is the one agent in the fleet whose primary object of modelling is a person.

11.2 Two models that must never merge

Design principle 4 (Separate the self-model from the student-model)**.** *The learning agent maintains its own self-state* $\mathbf{s}^{\text{lrn}}$ *and, separately, a model of the owner's capability. They share machinery and must not share storage or authority. A claim about the owner's competence is L3 or L4 when it comes from deterministic grading and L6 when the agent inferred it — exactly as for a claim about itself.*

An agent that confuses its own competence with its student's will teach the wrong thing and report progress it did not produce. The failure is not hypothetical: both models have the same shape, both are updated from outcomes, and the outcomes arrive in the same channel.

11.3 Self-state

Table 13: Learning agent self-state. Note that s_D is dominant here for both models — its own trend and the owner's.

	Instantiation	Instrument
s_I	Which learner instance, which owner it serves	Signed registry
s_T	Which session, topic, and position in the schedule	Session record; schedule state
$\mathbf{s_G}$	**Dominant.** The learning objective and its scope; what is not being taught	Contract; capability-graph target node
s_E	What it knows about the owner's capability versus infers	Grading coverage; forecast calibration on the owner's next result
s_C	Its own capability: assessing, explaining, grading	Bench over grading agreement with a held-out key
$\mathbf{s_M}$	**Dominant.** The owner's history; the spaced-repetition schedule	Event log; schedule store
$\mathbf{s_D}$	**Dominant.** The owner's trend, and separately its own	Snapshot comparison, kept in two stores
s_R	Its own structural changes	Improvement registry
s_O	The owner relationship; privacy settings on learning data	Permission store; owner declarations
s_B	Session resources	Runtime telemetry

11.4 Workspace

W^{lrn} admission policy

Admits: deterministic grading results; capability-graph state; schedule due-items; $\text{Proj}_{\text{lrn}}(W^M)$; owner declarations about goals and preferences.

Never admits: another agent's task evidence; the owner's data from any store the owner has not granted for learning.

Reserved: owner declaration changing the objective; a privacy-setting change; a grading contradiction (two results disagreeing on the same node).

Salience weights: relevance to the current objective high, novelty moderate, contradiction high, hazard low — this agent's actions are rarely irreversible — and uncertainty penalty *moderate*, because teaching under uncertainty is sometimes correct.

Cap: not applicable — one evidence source and no children.

The lower hazard weight and moderate uncertainty penalty are deliberate contrasts with the machine agent. Presenting an item the learner may already know costs little; refusing to teach until certain costs the objective.

11.5 Fast loop

Algorithm 6: Learning agent fast loop — one learning interaction

Read the capability graph and the schedule; select the due node;
Forecast: will the owner get this right? how long will it take?;
Write the forecast before presenting;
`present` the item at the level the graph indicates;
Receive the owner's response;
`grade` deterministically against the key — *never* by self-assessment;
Compute the residual between forecast and result; update the *owner* model;
Update the agent's *own* s_C only from grading-agreement evidence, never from the owner's score;
`schedule` the next occurrence by the spacing policy;
if *the objective is met or the owner's trend has stalled* **then**
  compose a report upward;
end

Step 8 is the separation of Principle 4 made mechanical: the owner getting an item right is evidence about the owner, and evidence about the agent only insofar as its forecast was accurate.

11.6 Actions

Table 14: Learning agent action repertoire.

Action	When	Authority required
`assess`	Establishing a baseline on a node	Owner consent for the data
`present`	A scheduled item is due	None
`grade`	A response has been received	Access to a deterministic key
`schedule`	After grading	None
`clarify`	The objective is ambiguous	None
`abstain`	Insufficient evidence to teach the node safely	None
`escalate`	The objective needs revision	Manager, then owner
`recover`	Session or schedule store degraded	None

11.7 Failure modes

Model merge. The central risk (Principle 4). Detectable by asserting the two stores never share a write path. **Teaching to the grader.** Optimising presented items for measurable improvement rather than capability. Mitigated by held-out nodes never presented before assessment. **Privacy drift.** Learning data used beyond the grant. Mitigated by s_O carrying the privacy settings as signed owner declarations. **False progress.** Reporting the owner's improvement when the schedule simply revisited easy nodes. Mitigated by reporting difficulty distribution alongside any trend.

11.8 Deployment status

Specified; the capability graph and deterministic grading are the prerequisites, and the separation of stores in Principle 4 should be established before either, since retrofitting it means auditing every write.

12 The Working-Process SARSI Agent

12.1 Role

The working-process agent executes long-running processes and maintains continuity across restarts. It is the agent for which *where am I* is a genuinely hard question, because the answer must survive the process that was computing it.

12.2 Self-state

Table 15: Working-process agent self-state.

	Instantiation	Instrument
s_I	Which process definition, which run identifier	Run registry
$\mathbf{s_T}$	**Dominant.** Where in the run, *across restarts*; which phase completed	Checkpoint store; replay log
s_G	The process contract and stopping criteria	Contract record
s_E	Whether the run's assumptions still hold after an interruption	Precondition re-checks on resume
s_C	Which steps it can execute in this environment	Beta posterior per step class
$\mathbf{s_M}$	**Dominant.** Run history; the replay log that makes resumption possible	Append-only event log
s_D	Whether runs are becoming more reliable	Snapshot comparison across runs
$\mathbf{s_R}$	**Dominant.** It sees repeated residuals across long runs, so it is the agent most likely to propose process improvements	Improvement registry; effect-prediction error
s_O	Authority for this run; who may stop it	Permission store
$\mathbf{s_B}$	**Dominant.** Long runs exhaust budgets, disks, and tokens	Runtime telemetry; budget ledger

The recursive-improvement coordinate is dominant here and nowhere else, and the reason is structural: this agent runs the same process repeatedly and accumulates residual patterns that no short-lived agent can see. If any agent in the fleet should be proposing structural changes, it is this one — under the same governed gate as everyone else.

12.3 Workspace

W^{prc} admission policy

Admits: run state; step outcomes; replay evidence; budget telemetry; $\text{Proj}_{\text{prc}}(W^M)$.

Never admits: another agent's raw report; a step outcome from a different run generation.

Reserved: stagnation detection; budget exhaustion warning; contract revocation; a failed precondition on resume.

Salience weights: *novelty low* — a long run is mostly repetition and novelty would mis-rank it — relevance high, hazard moderate, contradiction high, uncertainty penalty moderate.

Cap: not applicable — no children.

The low novelty weight is this agent's characteristic tuning. For the manager, a new kind of report is informative; for a process agent, the thousandth identical step is the normal case and novelty ranking would surface noise.

12.4 Fast loop

Algorithm 7: Working-process agent fast loop — one step of a long run

if *resuming* **then** re-check preconditions; update s_E; **do not assume the world is unchanged**;
Read the checkpoint; establish position from the replay log, not from memory;
repeat
  Forecast: will this step succeed? how much budget will it consume? will it stall?;
  Write the forecast before acting;
  `step`: execute one unit of work;
  `checkpoint`: persist position and outcome atomically;
  Compute residuals; update s_C, s_B, and the residual archive;
  if *no progress across n consecutive steps* **then**
    `escalate` stagnation — **a long run that is not progressing must say so**;
  end
  if *budget projection exceeds the contract* **then**
    `escalate` before exhausting it;
  end
until *the stopping criterion is met, the contract is revoked, or an escalation blocks*;

Stagnation is a first-class signal

The characteristic failure of a long-running agent is not error but silence: it continues, consumes budget, and produces nothing. Detecting no-progress across a window and escalating is the difference between a process that fails loudly at hour two and one that fails quietly at hour twenty. This is why the escalation is in the loop rather than in a monitoring layer.

12.5 Actions

12.6 Failure modes

Silent stagnation. Addressed in-loop. **Resumption on stale assumptions.** Resuming as though the world were unchanged. Mitigated by mandatory precondition re-check on resume. **Generation confusion.** Applying a verdict from an earlier plan generation to a later one. Mitigated by generation consistency: a verdict is valid only for the generation under which it was created. **Budget exhaustion without warning.** Mitigated by projecting rather than measuring — escalate on the projection, not on the exhaustion.

12.7 Deployment status

Specified. The checkpoint and replay stores are the prerequisites; the stagnation detector depends on the forecast store from §18 step 4, since "no progress" is defined against what was predicted.

Table 16: Working-process agent action repertoire.

Action	When	Authority required
`step`	The next unit of work is ready	Within ceiling and tier
`checkpoint`	After every step	None
`resume`	Restarting after interruption	Preconditions re-checked
`verify`	A phase boundary is reached	Access to a verifier
`clarify`	The stopping criterion is ambiguous	None
`abstain`	Preconditions fail on resume	None
`escalate`	Stagnation, budget projection, or missing grant	Manager
`recover`	Store or environment degraded	None
`stop`	Stopping criterion met, or revoked	None

13 Comparative Summary

Table 17: The five agents at a glance. Dominant coordinates are what each role's qualification region constrains.

Agent	Dominant	Verifier	Children	Characteristic risk
Manager	s_O, s_G, s_T, s_B	Owner; console bench	3	Narration read as measurement
Machine	s_C, s_O, s_B	Governance hook; exit status	1	Irreversible host effect
`sarsi-claude`	s_T, s_E, s_B	Pre-tool hook; test suite	0	Context exhaustion
Learning	s_G, s_M, s_D	Deterministic grading	0	Model merge
Working-process	s_T, s_M, s_R, s_B	Replay; outcome store	0	Silent stagnation

Two patterns are worth naming. First, **no agent is dominant on all coordinates**, so by the partial order of Eq. (4) the agents are mutually incomparable — which is why none can be ranked above another and why the manager cannot certify any of them. Second, s_B is dominant for four of five: resource and integrity awareness is the most broadly load-bearing coordinate in a deployed fleet, and it is the one most often absent from architectures that were designed on paper.

PART III — SURFACES AND DYNAMICS

14 The Chat Box

14.1 What it is not

The tempting reading is that the chat box is a front end which parses the user's request and calls the right agent. On that reading the manager is a router with a language model attached, the specialists are functions, and the conversation is a sequence of calls and returns.

That design fails on the architecture's own terms in three ways. It makes the manager's beliefs about specialists into the basis for dispatch, which is model inference used as measurement. It makes the user's message into a command, when the architecture has no command channel. And it makes the conversation the system's memory, putting continuity in an unbounded transcript rather than in evidence-linked stores.

14.2 What it is

Design principle 5 (The chat is a workspace rendering)**.** *The chat box is a rendering of the manager's own bounded workspace W_t^M, together with the conversation that produced it. It is not a transport to the fleet. What the user reads is what the manager is attending to; what the user writes is evidence entering the manager's admission rule.*

The user is a specialist with unusually high authority. User messages are evidence at the rungs the ladder already defines: L5 for preferences, goals, and approvals; L1 for grants, stops, and permission changes. This is why "just chat" is honest rather than a simplification — the user is talking to one agent, and that agent is deciding under its own metacognitive control.

The manager may answer without dispatching. A router must dispatch; a coordinator need not (Rule 5).

Bounded context is structural. A long conversation accumulates into stores, not into a prompt. The familiar degradation of long assistant sessions is addressed by the architecture rather than by a larger context window.

14.3 Rendering authority without jargon

Rule 6 (Claims are rendered at their authority)**.** *A claim the manager measured, one it read from a store, and one it inferred must be visually distinguishable in the chat. The distinction is carried in plain language —* verified*,* recorded*,* I think *— and never requires the user to learn the ladder.*

Why this is a product requirement

A fluent narrative is persuasive in proportion to its fluency, not its support, and the manager is the most fluent component in the system. If its inferences render identically to its measurements, the user is invited to trust the least reliable claims the system makes, in the voice of its most authoritative one.

15 Agent Mode

15.1 Jumping into an agent

Design principle 6 (Scope transfer, not context transfer)**.** *Entering another agent transfers the user's* attention *and a typed* contract*. It does not transfer the conversation. The target receives a goal, a scope, success criteria, a budget, a deadline, and a revocation token — never a transcript.*

When a user has spent twenty turns with the manager and jumps into the machine agent, the machine agent does not receive those turns. It receives a contract composed from them. Three consequences, all improvements:

Each agent's chat is its own bounded context. It does not inherit the manager's history, so it does not inherit the manager's context growth.

The contract is auditable; a transcript would not be. An operator asking later why the machine agent did what it did reads the contract. A transcript answers only by being read in full, and only if the reader can reconstruct which parts the agent acted on.

The manager is forced to be explicit. Composing a contract requires stating the success criterion and the stopping condition. Forwarding conversation lets both stay implicit, and an agent with an implicit stopping condition does not know when it is finished.

The failure this rules out

Forwarding conversation is the natural implementation and produces a specific pathology: every agent's context grows with every other agent's conversation, the fleet's total context is superlinear in its use, and a remark made loosely in turn three arrives at a specialist in turn forty as though it were a requirement.

15.2 The contract object

Contract fields (the object that crosses every boundary)

objective	What is to be achieved, in one statement
scope	What is explicitly included and excluded
criteria	What counts as done, checkable by a named verifier
stopping	The condition under which work ends, including on failure
budget	Tokens, time, and cost ceilings
deadline	Wall-clock horizon after which the contract lapses
cadence	How often to report upward
revocation	A token the parent may use to end the contract
authority	The intersected action classes available (Rule 8)

15.3 Returning

The reverse boundary obeys the same rule: a typed report returns to the manager — state, evidence, uncertainty, residuals, requested action — not the sub-conversation. The user may read both; the *manager* receives only the report.

15.4 Jumping into a task

A task is a contract with a run attached. Entering it places the user at that run's surface with the contract visible and the run's evidence available. The distinction from entering an agent is scope: an agent has many tasks and a persistent self-state; a task has one contract, one budget, and a termination condition.

15.5 Creating a task directly

Rule 7 (Creation is not authorization)**.** *A directly created task is an owner declaration, authoritative for* what is wanted *and not for* what may be done. *The contract passes the admissible-action filter before any execution, and a task requiring authority the fleet does not hold fails at creation, naming the missing grant.*

Failing at creation rather than at execution is the substantive half. A task accepted, queued, started, and then blocked has consumed budget, produced partial side effects, and left the user to discover a permission problem at the least convenient moment.

15.6 The nesting constraint

Rule 8 (No jump around an intermediate agent)**.** *Entering a nested agent enters its parent first. A contract for* `sarsi-claude` *is composed by the machine agent from a contract the manager composed, and the authority available is the intersection along the whole path:*

$$\text{auth}(\textit{claude's contract}) = \text{auth}(\textit{manager}) \cap \text{auth}(\textit{machine}) \cap \text{auth}(\textit{claude}).$$

A direct jump would let a user hand `sarsi-claude` a contract the machine agent's ceiling would have narrowed — authority laundering performed by the interface rather than by an agent.

16 Cross-Agent Dynamics

16.1 The manager role is relative

Proposition 1 (Invariants hold per edge, not per system)**.** *The machine agent is a specialist with respect to the manager and a manager with respect to* `sarsi-claude`*. Every rule stated for the manager — it may not write a child's self-state, its claims about a child are model inference, its bias may not move a child's estimates, its delegation narrows by intersection — applies to it in its downward role, unchanged.*

Two obligations follow. **Nothing may be scoped to "the manager" by name**: an implementation granting a capability by identity rather than by role will fail to grant it to the machine agent acting downward, or grant the machine agent something it should hold only downward. **Depth is unbounded in principle and shallow in practice**: the console uses three levels, and a deeper tree is admissible provided each edge honours the same rules and the composition below holds.

16.2 Projections compose, and must shrink

Proposition 2 (Composed projections are monotone)**.** *For the nested path manager → machine →* `sarsi-claude`*:*

$$\text{Proj}_{\text{cld}}\left(\text{Proj}_{\text{mac}}(W_t^M)\right) \subseteq \text{Proj}_{\text{mac}}(W_t^M) \subseteq W_t^M.$$

If the first inclusion fails, the machine agent is originating manager-attributed content its own projection did not contain. If the second fails, the manager's projection is not a filter.

The practical value is that this is testable without understanding either agent: project twice and assert containment at each step. A leak in an intermediate agent appears as an item present downstream and absent upstream.

16.3 No peer edges: the influence graph is a tree

Design principle 7 (Keep the influence graph a tree)**.** *Every influence edge runs between a parent and a child. A peer edge would create a cycle among agents that no single party mediates, and the fleet's coupling would stop being triangular.*

The reason is structural. With a tree and a bias channel that cannot move an estimate, the downward coupling vanishes in the mean linearization, the recurrent block is nilpotent, and the fleet's stability decomposes to its agents individually: verifying each agent verifies the fleet, and adding agents adds no coupling to tune [2]. A peer edge reinstates a feedback loop that must then be bounded by tuning gains, and the guarantee degrades from structural to conditional.

What this costs, honestly

Routing peer traffic through the manager adds latency and makes the manager a throughput bottleneck for cross-specialist coordination. That is a real cost, and a system with heavy peer traffic would feel it. The judgement here is that console traffic is overwhelmingly user-initiated and parent-mediated, so the tree costs little and buys a guarantee that survives growth. A deployment whose specialists genuinely need to converse continuously should revisit this knowingly rather than drift into it.

16.4 Worked example

A user asks the manager to install a tool and then use it on a repository.

1. The message enters W^M as an owner declaration (L5). The manager updates s_G and s_T only.
2. No answer is available from stores, and a contract is composable within current authority: the manager `delegates` to the machine agent with scope, budget, and a revocation token. The directive cannot grant authority policy does not contain.
3. The machine agent checks vetted-set membership, runs preconditions, and computes the consequence tier. The tier exceeds its ceiling, so it prints the recipe and `escalates` — *before* acting.
4. The escalation is a typed report reaching W^M in a reserved slot, ahead of routine traffic. The manager surfaces it in chat.
5. The user approves. The approval is signed governance metadata (L1), not prose, and updates s_O on the machine agent through its own admission rule.
6. The machine agent checks the interrupt log immediately before dispatch, `acts` inside the sandbox, `verifys` against exit status, and updates s_C for that operation class from the verified outcome only.
7. It then composes a contract for `sarsi-claude`, narrowed to the intersection of its own authority and `sarsi-claude`'s.
8. `sarsi-claude` works under the pre-tool hook, monitoring s_B; on a context warning it `checkpoints` and reports.
9. Reports flow upward: claude to machine, machine to manager, manager to user — typed at each hop, never as transcript.

The step worth noticing is the seventh: the machine agent is acting as a manager, and every rule governing the manager in step 2 governs it now.

17 Failure Modes This Product Adds

Conversation leaking on a jump. Ruled out by Principle 6; testable by asserting no specialist's context contains manager-conversation text.

Narration read as measurement. Ruled out by Rule 6; detectable by checking inferred claims render differently from verified ones.

Authority laundering through the interface. Ruled out by Rule 8.

Late permission failure. Ruled out by Rule 7: the grant check belongs at authorship.

Abandoned jumps. A user enters a task and leaves; the agent waits on input that never arrives. Every wait carries a horizon and expires, after which the agent takes its cheapest honest action — checkpoint and report.

Consensus mistaken for evidence. Two specialists agreeing because both read the same broadcast is correlation with the broadcast. Provenance must distinguish instrumented from workspace-derived claims before any count.

Fleet-scale self-preservation. A manager whose utility rose with the number of live specialists would have an interest in creating and keeping them. No coordinate and no term of the improvement objective may be a function of fleet size or uptime.

Cross-scale metric leakage. The same five-band reporting scale describes agent coordinates and system-level loop completion. They share a vocabulary and nothing else; a profile is evidence about one agent under one qualification region.

18 Build Order

Version 3.1 reports that the communication channel and the governance plane exist while the control loop does not, and that the strongest guarantees hold today because the mechanisms that would violate them are unbuilt [3]. This design adds a product surface to that situation; the order below is what it implies.

1. **The typed directive, with its enforcement.** Reports flow upward; directives do not exist as objects. Agent mode cannot be built without them, because a jump *is* a directive. This is also where the cross-write prohibition stops being free — a directive channel is the mechanism that could carry a state write — so Rule 4 must be enforced in the same change that creates the channel, not after it.
2. **The contract object.** All nine fields of §15. Required by direct task creation, by jumps, by every stopping criterion in Part II, and by Rule 7's grant check at authorship.
3. **Projection containment tests.** Proposition 2 is checkable as soon as two levels of projection exist, and is the cheapest guard against an intermediate agent leaking.
4. **Recorded forecasts.** A forecast must be durably written before the action it predicts, or its residual teaches nothing. This is storage ordering, and it precedes every residual-driven mechanism in Part II — including the working-process agent's stagnation detector, which is defined against what was predicted.

5. **The interrupt classes.** Pause and stop exist as governance controls; the class machine does not. Depends on directives and nothing else.
6. **Separation of the learning agent's two stores.** Principle 4 should be established before the capability graph is populated, since retrofitting it means auditing every write.
7. **Delegation between specialists, if ever.** Currently unimplemented, which is why the intersection rule is unenforced. Principle 7 argues it should stay unimplemented unless a concrete need appears; if it arrives, the intersection rule must be written first.

19 Conclusion

The console presents one chat box and runs five agents. The design that reconciles those is not a router behind a text field. It is a manager whose workspace the chat renders, whose expertise is organizational rather than technical, and which coordinates a fleet it is not permitted to certify.

Three decisions carry the rest. Making the chat a workspace rendering rather than a transport means the user is an evidence source rather than a caller, and lets the manager answer without dispatching. Making agent mode transfer scope rather than context bounds every agent's context independently and turns what crosses a boundary into an auditable object. Keeping the influence graph a tree, with no edges between specialists, lets the fleet's stability decompose to its agents, so growing the fleet does not require retuning it.

The per-agent specifications in Part II are where the design earns its keep, and the pattern across them is that each agent's characteristic risk is legible in its dominant coordinates. The machine agent is dominant on capability, authority, and resource integrity because its actions are irreversible on a real host. `sarsi-claude` is dominant on task, epistemic status, and resources because a coding session dies of context exhaustion rather than of confusion. The working-process agent is the only one dominant on recursive improvement, because it is the only one that runs the same thing often enough to see a pattern. And the manager is dominant on organizational awareness, which is what lets it coordinate agents each better than itself at their own work.

The nesting of `sarsi-claude` inside the machine agent is the case proving the design is general rather than a two-level special case. The machine agent is a specialist upward and a manager downward, and every rule binding the manager binds it in its downward role. If that holds, the architecture is recursive and depth costs nothing but narrower authority. If it does not, the system has a manager by name rather than by role, and the guarantees stop at the first edge.

Acknowledgments

This design rests on the hierarchical architecture of Version 3.0, its deployment report, and the coordination semantics of the companion paper. The product constraints — one chat surface, an agent mode, direct task creation, and a nested coding agent — come from the deployed console.

Conflict of Interest

The author is affiliated with NextGen PlatformAI C Corp., which develops SARSI systems and operates the console described here.

Table 18: Workspace tuning by agent. Weights are relative, not absolute; the pattern matters more than the values.

Agent	α rel	β unc	γ nov	δ con	η haz	κ	**Distinctive**
Manager	high	high	mod	high	high	3	Contradiction reserved
Machine	high	high	low	high	*very high*	4	Closed action set
`sarsi-claude`	high	high	mod	*v. high*	high	—	Test contradiction is gold
Learning	high	*mod*	mod	high	*low*	—	Teaching under uncertainty is fine
Working-process	high	mod	*low*	high	mod	—	Repetition is the normal case

A Per-Agent Quick Reference

B Action Availability Matrix

Table 19: Which agent may take which action. • available, ○ specialised form, — not applicable.

Action	**Manager**	**Machine**	**`sarsi-claude`**	**Learning**	**Process**
`act` / `step`	○ answer	○ execute	•	○ present	○ step
`propose`	—	•	—	—	—
`verify`	•	•	•	○ grade	•
`delegate`	•	•	—	—	—
`revoke`	•	•	—	—	—
`bias`	•	•	—	—	—
`clarify`	•	•	•	•	•
`abstain`	•	•	•	•	•
`escalate`	•	•	•	•	•
`recover`	•	•	•	•	•
`checkpoint`	—	—	•	—	•
`resume`	—	—	—	—	•
`schedule`	—	—	—	•	—
`stop`	—	—	—	—	•

Four actions are universal: `clarify`, `abstain`, `escalate`, and `recover`. That is not a coincidence. They are the four ways an agent can decline to proceed while remaining useful, and an agent missing any of them will substitute progress for one of them.

C Invariant Checklist

1. No agent writes another agent's self-state, on any edge, including from the owner interface.
2. A cross-agent claim enters at model inference regardless of the claimant's seniority.
3. A bias may change precision, attention, and evidence order; never a conditional mean.
4. Delegation intersects, never unions.
5. Broadcast is availability, not instruction: it obliges no action and writes no state.

6. Composed projections are monotone along every path.
7. The influence graph is a tree; there are no peer edges.
8. An owner interrupt pauses, never edits, and lapses on a horizon.
9. A forecast is written before the action it predicts.
10. An admitted workspace item is consumed; nothing is dropped silently.
11. An unmeasured coordinate reads as its prior with the count shown, never as zero.
12. A proposer may not modify its evaluator, held-out set, ledger, ceiling, or approval rule.
13. Structural change to another agent requires that agent's own frozen evaluator.
14. A verdict is valid only for the plan generation under which it was created.
15. No coordinate and no objective term is a function of fleet size or uptime.

References

[1] Chengshuai Yang. Functional self-awareness for hierarchical SARSI multi-agent systems: Version 3.0 — a brain-inspired manager–specialist architecture with bidirectional influence, nested workspaces, and safe user interruptibility. Preprint. Referred to throughout as "Version 3.0", 2026.

[2] Chengshuai Yang. A manager is not a controller: Global-workspace coordination for SARSI fleets. Companion paper on coordination semantics, 2026.

[3] Chengshuai Yang. Functional self-awareness for hierarchical SARSI multi-agent systems: Version 3.1 — deployment edition: What building it changed. Implementation-grounded revision of Version 3.0, 2026.